

Homework 4: Global Placement

Name: 陸明德

Student ID: 111062693

The wirelength and the runtime of each testcase

testcase	wirelength	runtime	status
public1	87987694	23.53	success
public2	18794771	49.74	success
public3	746977918	93.75	success

The details of your algorithm.

Implementation of evaluateF and evaluateFG

First of all, I learned the algorithm and implementation from "Chris Chu. Chapter 11: Placement. In Electronic Design Automation: Synthesis, Verification, and Test, Laung-Terng (L.-T.) Wang, Yao-Wen Chang and Kwang-Ting (Tim) Cheng, Editors, Elsevier, 2009.". [link](#)

I choose Log-sum-exponential as my wirelength function. The reason is that it is a convex function, so it is easy to find the minimum. And it is also a smooth function, so it is easy to calculate the gradient.

Since HPWL can be approximated by the log-sum-exponential, below is the implementation of log-sum-exponential.

```
f = 0.0; // objective cost function : LSE wirelength + Bin Density
memset(WL, 0.0, sizeof(double) * numModules * 4);
// LSE wirelength
for(int i = 0; i < numModules; ++i){
    WL[4*i] = exp(x[2*i] / alpha);
    WL[4*i+1] = exp(-x[2*i] / alpha);
    WL[4*i+2] = exp(x[2*i + 1] / alpha);
    WL[4*i+3] = exp(-x[2*i + 1] / alpha);
}

for(unsigned int i = 0; i < _placement.numNets(); ++i){
    double sumX1 = 0.0, sumX2 = 0.0, sumY1 = 0.0, sumY2 = 0.0;
    for(unsigned int j = 0; j < _placement.net(i).numPins(); ++j){
        int mID = _placement.net(i).pin(j).moduleId();
        sumX1 += WL[4*mID];
        sumX2 += WL[4*mID+1];
        sumY1 += WL[4*mID+2];
        sumY2 += WL[4*mID+3];
    }
}
```

```
f += alpha*(log(sumX1) + log(sumX2) + log(sumY1) + log(sumY2));
}
```

Since each module have to caculate the exp() function, it is time consuming. So I use the variable "WL" to store the exp() value of each module position. Including positive and negative x, y direction. So I can save a lot of time to calculate the exp() function.

The gradient of log-sum-exponential is below.

```
... same as above

for (unsigned int j = 0; j < _placement.net(i).numPins(); ++j){
    int mID = _placement.net(i).pin(j).moduleId();
    g[2*mID] += (alpha * (WL[4*mID] / sumX1 - WL[4*mID+1] / sumX2));
    g[2*mID+1] += (alpha * (WL[4*mID+2] / sumY1 - WL[4*mID+3] / sumY2));
}
```

The gradient of log-sum-exponential is easy to cacualte by the equation in the wiki page. [link](#)

The function of bin density is below.

```
// Bin Density, bell-shaped smoothing function
memset(binDensity, 0.0, sizeof(double) * numBins);
double Wb = 0.0, Hb = 0.0, c = 0.0;
double thetaX = 0.0, thetaY = 0.0, dX = 0.0, dY = 0.0, aX = 0.0, bX = 0.0, aY = 0.0, bY = 0.0, ABSdX = 0.0, ABSdY = 0.0;

for (int a = 0; a < binCut; ++a){
    for (int b = 0; b < binCut; ++b){
        for (int i = 0; i < numModules; ++i){
            Wi = _placement.module(i).width(); Hi = _placement.module(i).height();
            c = _placement.module(i).area()/binArea;
            // dX: center-to-center distance of the block v and the bin b in the
x-direction.
            dX = x[2*i] - ((a + 0.5)*Wb + _placement.boundaryLeft());
            // distance should be positive
            ABSdX = abs(dX);
            // dY: center-to-center distance of the block v and the bin b in the
y-direction.
            dY = x[2*i + 1] - ((b + 0.5)*Hb + _placement.boundaryBottom());
            // distance should be positive
            ABSdY = abs(dY);
            aX = 4 / ((Wb + Wi) * (2 * Wb + Wi));
            bX = 4 / (Wb * (2 * Wb + Wi));
            aY = 4 / ((Hb + Hi) * (2 * Hb + Hi));
            bY = 4 / (Hb * (2 * Hb + Hi));

            if(ABSdX <= Wi/2.0 + Wb/2.0) thetaX = (1 - aX * ABSdX * ABSdX);
            else if(ABSdX <= Wi/2.0 + Wb) thetaX = (bX * pow(ABSdX - (Wb + Wi /
```

```

    2), 2));
        else thetaX = 0;

        if(ABSdY <= Hi/2.0 + Hb/2.0) thetaY = (1 - aY * ABSdY * ABSdY);
        else if(ABSdY <= Hi/2.0 + Hb) thetaY = (bY * pow(ABSdY - (Hb + Hi /
    2), 2));
        else thetaY = 0;

        binDensity[a + binCut*b] += c * thetaX * thetaY;
    }
    f += beta * pow(binDensity[a + binCut*b] - targetDensity , 2);
}
}

```

The implementation of bin density is from the book I mentioned above. The variable's name is the same as the book. The only difference is that I use the absolute value of dX and dY. Because the distance should be positive.

Here c is a normalization factor.

And the gradient of bin density is below.

```

for (int a = 0; a < binCut; ++a)
{
    for (int b = 0; b < binCut; ++b)
    {
        for (int i = 0; i < numModules; ++i)
        {
            ... same as above

            if(ABSdX <= Wi/2.0 + Wb/2.0) grad[2*i] = c * -2 * aX * dX * thetaY;
            else if(ABSdX <= Wi/2.0 + Wb){
                if(dX >0) grad[2*i] = c * 2 * bX * (dX - (Wb + Wi / 2)) * thetaY;
                else grad[2*i] = c * 2 * bX * (dX + (Wb + Wi / 2)) * thetaY;
            }
            else grad[2*i] = 0;

            if(ABSdY <= Hi/2.0 + Hb/2.0) grad[2*i + 1] = c * -2 * aY * dY *
thetaX;
            else if(ABSdY <= Hi/2.0 + Hb){
                if(dY >0) grad[2*i + 1] = c * 2 * bY * (dY - (Hb + Hi / 2)) *
thetaX;
                else grad[2*i + 1] = c * 2 * bY * (dY + (Hb + Hi / 2)) * thetaX;
            }
            else grad[2*i + 1] = 0;
        }
        f += beta * pow(binDensity[a + binCut*b] - targetDensity , 2);

        for (int i = 0; i < numModules; ++i)
        {
            g[2*i] += 2 * beta * (binDensity[a + binCut*b] - targetDensity) *

```

```

grad[2*i];
    g[2*i + 1] += 2 * beta * (binDensity[a + binCut*b] - targetDensity) *
grad[2*i + 1];
    }
}
}

```

The gradient of bin density is caculated by the derivative of the bin density function.

The papers algorithm and my implementation difference

I used bin density function result to determine my beta. Instead of using the constant beta, I used the beta that can dynamically change. The reason is that the constant beta may cause the result to be stuck in the local minimum. So I use the bin density function result to determine the beta.

It's very similar to the paper's Uzawa algorithm . The implementation is below.

```

void ExampleFunction::Increase_Lambda(double alpha)
{
    beta += DF * alpha;
}

```

The alpha is a user defined parameter. It is used to control the speed of beta increasing.

The rest of the algorithm is the same as the paper. Using the analytical methods to calculate the gradient and find the minimum solution.

Try your best to enhance your solution quality.

The way I enhance the solution quality

I tried a lot of different parameters to find the best solution.

First of all, I tried to change the alpha and beta. I thought alpha is better to be small, because it can make the wirelength more accurate. However, if alpha is too small, the final wirelength will be too large. So I tried to change the alpha to boundryWidth / 100, 50, 20, 10, 4, 2, 1, . And I found that alpha = boundryWidth / 4 is the best. Because it can make the wirelength more accurate and the final wirelength is not too large.

Then I tried to change the beta. I thought beta is better to be large, because it can make the bin density more accurate. However, if beta is too large, the bin density will be too large. Cause the wirelength is not that important. So I tried to change the beta to 0.1, 0.5, 1, 2, 5, 10, 20, 50, 100. And I found that used bin density function result to determine my beta is better. Instead of using the constant beta, I used the beta that can dynamically change.

Besides, I thought that iteration of gradient descent is better to be large. Because it can find the minimum solution more accurate. However, the result was opposite at all.

Please compare your results with the previous top 5 students' results and show your advantage in solution quality.

Rank	public1	public2	public3
1	68783367	8778312	456197589
2	79542407	9155930	478967869
3	83860394	9783498	463664700
4	73804994	11105419	413161709
5	80176617	10921603	539864787
My	87987694	18794771	746977918

what do you think that you could do to improve the result in the future?

The initial solution is very important. If the initial solution is not good, the final solution will be stuck in the local minimum. So I think the initial solution is the most important thing to improve the result in the future.

The other thing is that step size is also very important. If the step size is too large, the final solution will be stuck in the local minimum. If the step size is too small, the final solution will be too slow. Now I use the constant step size. I think it is better to use the dynamic step size like implemented by the line search. The line search is to find the best step size to find the minimum solution. By using the line search, the final solution will be more better.